



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ

КУРСОВ ПРОЕКТ

Дисциплина: Програмни езици

Изготвил: Велизара Стоянова Сотирова

Курс: III, КСИ

Група: 45

Фак. № 121223009

e-mail: vsotirova@tu-sofia.bg

Ръководител: Д. Вълков

София

2025

Съдържание

1	Задание	2
2	Въведение	3
3	Архитектура на системите	3
4	Потребителско ръководство.....	4
5	Техническа документация	7
6	ООП принципи в проекта	15
7	Възможности за разширение.....	16
8	Заключение	16

1 Задание

Създайте приложение, което да поддържа информация за доставчици на различни материали от оптиката. Работи се с доставчици, за които трябва да се съхранява техния Булстат, името и местонахождението им и телефонен номер. Всеки доставчик може да достави един или повече оптика, които се характеризират с тип, дебелина, диоптър и наименованието на материала. Приложението да има възможност за въвеждане на произволен брой различни доставчици и оптика.

Да има възможност за избор на доставчик и от него да се поръчват различни материали, като се изчислява общата цена на поръчката.

Класовете (най-малко 3 класа при реализацията) трябва да капсулира всичките детайли. Използват се private инстанции на променливите за съхраняване на различните детайли. Трябва да има най-малко gва конструктора, public getters/setters за private инстанции на променливите.

Необходимо е да извършвате проверка на входните данни.

Да се предефинира операцията <<, която да се използва за извеждане на данните. Данните да се четат и съхраняват във файл.

Класовете да се опишат с UML клас диаграма.

Задължително данните да се въвеждат динамично, чрез меню.

2 Въведение

1. Предназначение на системата

Optical Materials Management System е конзолно приложение, реализирано на C++ за управление на доставчици на оптични материали. Системата позволява регистриране на доставчици, управление на каталози с оптични материали и създаване на поръчки.

2. Основни функционалности

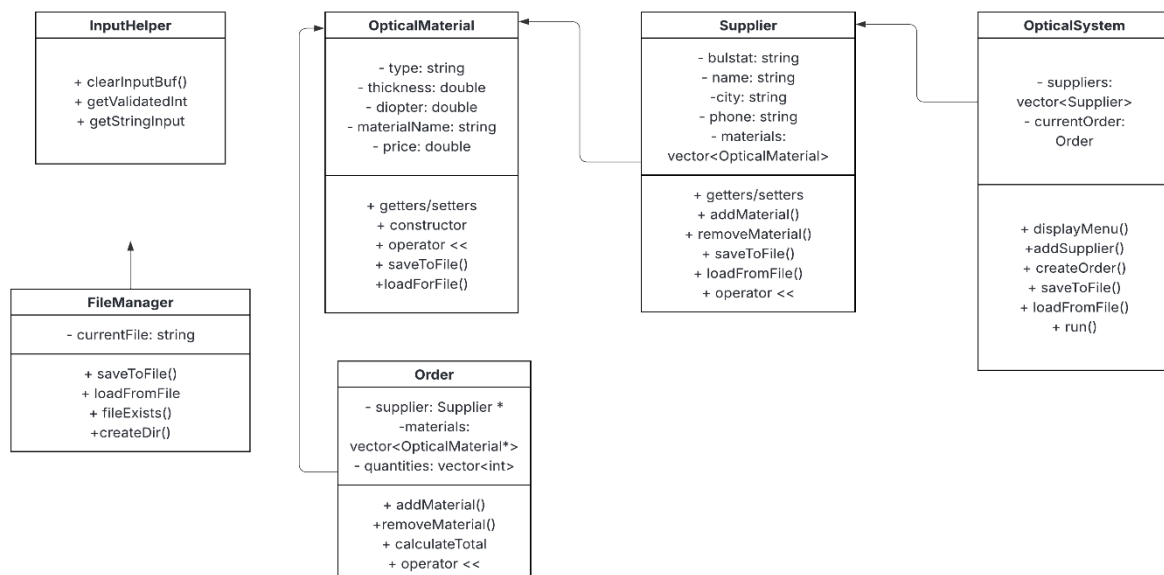
- Управление на доставчици (добавяне, преглед)
- Управление на оптични материали
- Създаване на поръчки
- Работа с файлове (запис/четене в различни формати)
- Валидация на входни данни

3. Цели на проекта

- Да се демонстрират ООП принципи в C++
- Да се реализира напълно функционална система за управление
- Да се включат всички изисквания от заданието

3 Архитектура на системите

1. UML Диаграма на класовете



Фиг. 1: UML диаграма

2. Връзки между класовете

- **Композиция:** Supplier съдържа vector<OpticalMaterial>

- **Агрегация:** Order съдържа указатели към OpticalMaterial
- **Зависимост:** OpticalSystem използва всички останали класове
- **Utility клас:** InputHelper и FileManager

3. Файлова структура

OpticalShopProject/

```
├── main.cpp           # Точка на влизане
├── OpticalSystem.h/cpp # Главен контролер
├── InputHelper.h/cpp   # Помощник за вход
├── OpticalMaterial.h/cpp # Оптични материали
├── Supplier.h/cpp      # Доставчици
├── Order.h/cpp         # Поръчки
└── FileManager.h/cpp   # Управление на файлове
```

4 Потребителско ръководство

1. Стартиране на програмата

```
=== OPTICAL MATERIALS MANAGEMENT SYSTEM ===
1. Add new supplier
2. Display all suppliers
3. Add material to supplier
4. Create order
5. Display current order
6. Calculate order price
7. File operations
8. Exit
Select option:
```

2. Управление на доставчици

2.1. Добавяне на нов доставчик

```
=== ADD NEW SUPPLIER ===
Enter VAT number: 123456789
Enter company name: OptiCorp
Enter city: Sofia
Enter phone: +359888123456
```

Валидации:

- VAT номер: 9-13 цифри, само цифри

- Име: непразен стринг
- Град: поне 3 символа, букви и допустими символи
- Телефон: минимум 5 цифри, може да съдържа +, - и интервали

2.2. Презлед на доставчици

```
=== ALL SUPPLIERS ===
[1] Supplier: OptiCorp
    VAT: 123456789
    City: Sofia
    Phone: +359888123456
    Materials (2):
      1. Type: Lens, Thickness: 1.5 mm, Diopter: -2.0, Material: Polycarbonate, Price: 25.50 BGN
      2. Type: Frame, Thickness: 2.0 mm, Diopter: 0.0, Material: Metal, Price: 45.00 BGN
```

3. Управление на материали

3.1. Добавяне на материал към доставчик

```
=== ADD MATERIAL ===
Enter material type: Lens
Enter thickness (mm): 1.5
Enter diopter: -2.0
Enter material name: Polycarbonate
Enter price: 25.50
```

Валидации:

- Дебелина: положително число
- Цена: неотрицателно число

4. Създаване на поръчки

4.1. Създаване на нова поръчка

```
=== CREATE ORDER ===
Select supplier number: 1
Supplier's materials:
1. Type: Lens, Thickness: 1.5 mm, Diopter: -2.0, Material: Polycarbonate, Price: 25.50 BGN
2. Type: Frame, Thickness: 2.0 mm, Diopter: 0.0, Material: Metal, Price: 45.00 BGN

Add material to order (0 to finish): 1
Enter quantity: 2
Material added to order!
```

4.2. Презлед на поръчка

```
=== CURRENT ORDER ===  
Order to: OptiCorp  
Order items:  
  1. Type: Lens, Thickness: 1.5 mm, Diopter: -2.0, Material: Polycarbonate, Price: 25.50 BGN x 2 = 51.00 BGN  
Total amount: 51.00 BGN
```

5. Файлови операции

5.1. File Operations меню

```
=== FILE OPERATIONS ===  
1. Save data to file  
2. Load data from file  
3. Check file information  
4. Create directory  
5. Return to main menu  
Select option:
```

5.2. Запазване на данни

```
=== SAVE DATA ===  
1. Save as text file (readable)  
2. Save as binary file (faster)  
Select format: 1  
Enter filename: suppliers.txt  
✓ Data saved successfully to: suppliers.txt
```

5.3. Зареждане на данни

```
=== LOAD DATA FROM FILE ===  
Enter filename to load: suppliers.txt  
✓ File found. Size: 1024 bytes  
  
1. Load as text file  
2. Load as binary file  
3. Preview file content (text only)  
Select option: 3
```

5.4. Преглед на файлово съдържание

```
=== FILE CONTENT PREVIEW ===
```

```
File: suppliers.txt
```

```
=====
```

```
1: 2
```

```
2: 123456789
```

```
3: OptiCorp
```

```
4: Sofia
```

```
5: +359888123456
```

```
...
```

```
=====
```

```
Total lines shown: 15
```

5 Техническа документация

1. Класове и тяхната роля

1.1. OpticalMaterial (Оптичен материал)

Роля: Представява един оптичен материал с всички характеристики.

Член-данни:

- type: Тип на материала (Lens, Frame, Coating)
- thickness: Дебелина в mm (Валидация: > 0)
- diopter: Оптична сила в диоптри
- materialName: Наименование на материала
- price: Цена в BGN (Валидация: ≥ 0)

Ключови методи:

- saveToBinary()/loadFromBinary(): Двоична сериализация
- saveToText()/loadFromText(): Текстова сериализация
- operator<<: Форматирано извеждане

1.2. Supplier (Доставчик)

Роля: Управлява информация за доставчик и неговите материали.

Член-данни:

- bulstat: ЕИК номер (9-13 цифри)
- name: Име на фирмата
- city: Град (поне 3 символа)
- phone: Телефонен номер (минимум 5 цифри)
- materials: Вектор от OpticalMaterial обекти

Ключови методи:

- addMaterial(): Добавя материал към доставчика

- `getMaterialsCount()`: Връща брой материали
- `getTotalMaterialValue()`: Изчислява обща стойност на материалите
- Статични валидационни методи за телефон и ЕИК

1.3. Order (Поръчка)

Роля: Представява поръчка към конкретен доставчик.

Член-данни:

- `supplier`: Указател към доставчика
- `materials`: Указатели към избрани материали
- `quantities`: Количества за всеки материал

Ключови методи:

- `addMaterial()`: Добавя материал с количество
- `calculateTotalPrice()`: Изчислява обща сума
- `clearOrder()`: Изчиства поръчката

1.4. InputHelper (Помощник за вход)

Роля: Предоставя помощни функции за обработка на потребителски вход.

Статични методи:

- `clearInputBuffer()`: Изчиства входния буфер
- `getValidatedInt()`: Взема валидирано цяло число
- `getValidatedDouble()`: Взема валидирано дробно число
- `getStringInput()`: Взема символен низ

1.5. FileManager (Управление на файлове)

Роля: Управлява операциите с файлове.

Поддържани формати:

- `BINARY`: Двоичен формат (по-малък, по-бърз)
- `TEXT`: Текстов формат (четим от хора)

Ключови методи:

- `saveToFile()`: Записва данни във файл
- `loadFromFile()`: Чете данни от файл
- `fileExists()`: Проверява дали файл съществува
- `createDirectory()`: Създава директория

1.6. OpticalSystem (Главна система)

Роля: Главният контролер и координатор на приложението.

Член-данни:

- `suppliers`: Всички регистрирани доставчици

- currentOrder: Текуща активна поръчка
- fileManager: Инстанция на FileManager

Ключови методи:

- run(): Главен цикъл на приложението
- addSupplier(): Добавя нов доставчик
- createOrder(): Създава нова поръчка
- loadData(): Зарежда данни от файл с preview

2. Файлови формати

2.1. Текстов формат

2	← Брой доставчици
123456789	← ЕИК на първи доставчик
OptiCorp	← Име
Sofia	← Град
+359888123456	← Телефон
2	← Брой материали
Lens	← Тип на материал 1
1.5	← Дебелина
-2.0	← Диоптър
Polycarbonate	← Име на материал
25.50	← Цена
Frame	← Тип на материал 2
2.0	← Дебелина
0.0	← Диоптър
Metal	← Име на материал
45.00	← Цена
...	← Втори доставчик и т.н.

2.2. Двоичен формат

[4 байта: брой доставчици]
[за всеки доставчик]:
[4 байта: дължина на ЕИК]
[ЕИК като низ]
[4 байта: дължина на име]
[име като низ]
... и т.н. за всички полета

3. Валидационна система

3.1. VAT номер валидация

```
// Валидация на VAT номер
while (true) {
    bulstat = InputHelper::getStringInput("Enter VAT number: ");

    // Проверка за празен вход
    if (bulstat.empty()) {
        cout << "VAT number cannot be empty! Please try again.\n";
        continue;
    }

    // Проверка за дължина
    if (bulstat.length() < 9 || bulstat.length() > 13) {
        cout << "Invalid VAT number! Please enter a number with length 9-13 symbols.\n";
        continue;
    }

    // Проверка дали всички символи са цифри
    bool allDigits = true;
    for (char c : bulstat) {
        if (!isdigit(static_cast<unsigned char>(c))) {
            allDigits = false;
            break;
        }
    }

    if (!allDigits) {
        cout << "Invalid VAT number! VAT must contain only digits (0-9).\n";
        continue;
    }

    break; // Успешна валидация
}
```

3.2. Валидация за име на фирмата

```
// Въвеждане на име
while (true) {
    name = InputHelper::getStringInput("Enter company name: ");

    // Премахваме водещи и завършващи whitespace символи
    size_t start = name.find_first_not_of(" \t\n\r\f\v");
    size_t end = name.find_last_not_of(" \t\n\r\f\v");

    if (start == std::string::npos) {
        cout << "Company name cannot be empty or contain only spaces! Please try again.\n";
        continue;
    }

    name = name.substr(start, end - start + 1);

    // Проверка за минимална дължина
    if (name.length() < 3) {
        cout << "Company name must be at least 3 characters long! Please try again.\n";
        continue;
    }

    // Проверка за разрешени символи: букви, цифри, интервали и тире
    bool validName = true;
    bool hasLetterOrDigit = false;

    for (size_t i = 0; i < name.length(); i++) {
        char c = name[i];

        // Проверка за разрешени символи
        if (std::isalnum(static_cast<unsigned char>(c))) {
            hasLetterOrDigit = true;
        }
        else if (c == ' ') {
            // Интервалите са позволени, но проверяваме за последователни интервали
            if (i > 0 && name[i - 1] == ' ') {
                cout << "Company name cannot have consecutive spaces! Please try again.\n";
                validName = false;
                break;
            }
        }
        else if (c == '-') {
            // Тиретата са позволени
            // Можем да добавим допълнителни проверки за тирета ако искаме:
            // - Не позволяваме тире в началото или края
            if (i == 0 || i == name.length() - 1) {
                cout << "Company name cannot start or end with a dash (-)! Please try again.\n";
                validName = false;
                break;
            }
        }
    }
}
```

```

// - Не позволяваме последователни тирета
if (i > 0 && name[i - 1] == '-') {
    cout << "Company name cannot have consecutive dashes (--)! Please try again.\n";
    validName = false;
    break;
}

// - Не позволяваме тире след интервал или интервал преди тире
if (i > 0 && name[i - 1] == ' ') {
    cout << "Company name cannot have space before dash! Please try again.\n";
    validName = false;
    break;
}

if (i < name.length() - 1 && name[i + 1] == ' ') {
    cout << "Company name cannot have dash before space! Please try again.\n";
    validName = false;
    break;
}
}

else if (c == '&') {
    if (i == 0 || i == name.length() - 1) {
        cout << "Company name cannot start or end with an ampersand (&! Please try again.\n";
        validName = false;
        break;
    }

    if (i > 0 && name[i - 1] == '&') {
        cout << "Company name cannot have consecutive ampersands (&&! Please try again.\n";
        validName = false;
        break;
    }

    if (i > 0 && name[i - 1] == ' ') {
        cout << "Company name cannot have space before ampersand! Please try again.\n";
        validName = false;
        break;
    }

    if (i < name.length() - 1 && name[i + 1] == ' ') {
        cout << "Company name cannot have ampersand before space! Please try again.\n";
        validName = false;
        break;
    }
}

else {
    // Всички други символи са забранени
    validName = false;
    break;
}
}

if (!validName) {
    // Съобщението вече е изписано в специфичните проверки
    continue;
}
}

```

```

if (!validName) {
    // Съобщението вече е изписано в специфичните проверки
    continue;
}

if (!hasLetterOrDigit) {
    cout << "Company name must contain at least one letter or digit! Please try again.\n";
    continue;
}

// Допълнителна проверка: не трябва да започва или завършва с интервал
if (name.front() == ' ' || name.back() == ' ') {
    cout << "Company name cannot start or end with a space! Please try again.\n";
    continue;
}

break; // Успешна валидация

```

3.3. Валидация за телефон

```
// Валидация на телефон
while (true) {
    phone = InputHelper::getStringInput("Enter phone number: ");

    // Проверка за празен вход
    if (phone.empty()) {
        cout << "Phone number cannot be empty! Please try again.\n";
        continue;
    }

    // Проверка за валидни символи и брой цифри
    bool validChars = true;
    int digitCount = 0;

    for (char c : phone) {
        if (isdigit(static_cast<unsigned char>(c))) {
            digitCount++;
        }
        else if (c != ' ' && c != '+' && c != '-') {
            validChars = false;
            break;
        }
    }

    if (!validChars) {
        cout << "Invalid phone number! Only digits, spaces, + and - are allowed.\n";
        continue;
    }

    if (digitCount < 5 || digitCount > 15) {
        cout << "Invalid phone number! Must be between 5-15 digits.\n";
        continue;
    }

    break; // Успешна валидация
}
```

3.4. Валидация за град

```
// Валидация на град
while (true) {
    city = InputHelper::getStringInput("Enter city: ");

    // Проверка за празен вход
    if (city.empty()) {
        cout << "City cannot be empty! Please try again.\n";
        continue;
    }

    // Проверка за минимална дължина
    if (city.length() < 3) {
        cout << "City name must be at least 3 characters long! Please try again.\n";
        continue;
    }

    // Проверка дали има само букви, интервали и тирета
    bool validCity = true;
    bool hasLetter = false;

    for (char c : city) {
        if (isalpha(static_cast<unsigned char>(c))) {
            hasLetter = true;
        }
        else if (c != ' ') {
            // Допускаме интервали
            validCity = false;
            break;
        }
    }

    if (!validCity) {
        cout << "Invalid city name! Use only letters and spaces.\n";
        continue;
    }

    if (!hasLetter) {
        cout << "City name must contain at least one letter! Please try again.\n";
        continue;
    }

    // По желание: Проверка дали започва с главна буква
    if (!isupper(static_cast<unsigned char>(city[0]))) {
        cout << "Note: City names usually start with a capital letter.\n";
        // Не го правим задължително, само предупреждение
    }

    break; // Успешна валидация
}
```

4. ООП принципи в кода

4.1. Кapsуляция (Encapsulation)

```
class Supplier {  
private: // СКРИТИ данни  
    std::string bulstat;  
    std::vector<OpticalMaterial> materials;  
  
public: // ПУБЛИЧЕН интерфейс  
    std::string getBulstat() const { return bulstat; }  
    void addMaterial(const OpticalMaterial& material);  
};
```

4.2. Абстракция (Abstraction)

```
// Потребителят не знае имплементацията  
order.calculateTotalPrice(); // Просто работи  
system.saveToFile();         // Просто запазва
```

4.3. Композиция (Composition)

```
class Supplier {  
    std::vector<OpticalMaterial> materials; // Supplier HAS-A materials  
};
```

4.4. Презареждане на оператори (Operator Overloading)

```
friend std::ostream& operator<<(std::ostream& os, const Supplier& s) {  
    os << "Supplier: " << s.name << "\n"  
        << "VAT: " << s.bulstat;  
    return os;  
}
```

6 ООП принципи в проекта

1. Кapsуляция

- Всички член-данни са private
- Достъп се осъществява чрез публични методи
- Валидация в setters

2. Абстракция

- Сложната логика е скрита зад прости интерфейси

- Потребителят работи с високо ниво на абстракция

3. Композиция и агрегация

- Supplier съдържа OpticalMaterial обекти (композиция)
- Order съдържа указатели към OpticalMaterial (агрегация)

4. Полиморфизъм

- Презареден оператор << за форматиран изход
- Единен интерфейс за различни типове операции

7 Възможности за разширение

4.1. Функционални разширения

1. История на поръчките - архивиране на стари поръчки
2. Справки и отчети - статистики и анализи
3. Потребителска система - различни нива на достъп
4. Графичен интерфейс

4.2. Технически разширения

1. База данни
2. Мрежова функционалност - клиент-сървър архитектура
3. Мултиезична поддръжка - интернационализация
4. Unit тестове

8 Заключение

1. Резюме на проекта

Optical Materials Management System е функционално приложение, което демонстрира солидно разбиране на ООП принципи в C++. Системата успешно решава поставените задачи чрез добре структурирана архитектура и надеждна валидация.

2. Технически принос

- Архитектура с ясни отговорности
- Надеждна валидация на входни данни
- Гъвкава файлова система с поддръжка на multiple формати
- Ефективно управление на паметта чрез STL контейнери

3. Бъдещо развитие

С добавени разширения системата има потенциал да се превърне в професионален софтуерен продукт с пазарна стойност.